AWS-POWERED

# Image-to-Meme Matching Platform

## Technical Project Report & Implementation Blueprint

**PROJECT STATUS** Dataset uploaded to S3. The metadata prefix exists. Rekognition preprocessing, API/backend, and frontend are not yet confirmed as executed or deployed.

**Version:** 1.0

**Date:** 26 October 2026

**Primary platform:** Amazon Web Services (AWS)

**Current dataset location:** s3://my-meme-database-2026/images/

**Current bucket region:** US East (N. Virginia), us-east-1; inferred from LocationConstraint=null

Purpose: explain the concept, architecture, matching logic, AWS services, security, costs, implementation order, and demo narrative.

# 1. Executive Summary

The Image-to-Meme Matching Platform is a web application for a college community. A student opens the website, captures or selects a selfie, and receives a meme chosen from a curated collection. The first version uses Amazon Rekognition to identify visible facial attributes and expressions in the student image, then compares them with attributes precomputed for the meme collection. A lightweight scoring algorithm ranks candidates and chooses one of the best matches.

The key architectural principle is to separate one-time catalogue preprocessing from live user requests. The meme dataset is analyzed once and saved as metadata in S3. For each new student submission, the backend analyzes only that student image, reads the prepared metadata, scores candidates, and returns a temporary URL for the selected meme.

> **IMPORTANT SCOPE DISTINCTION** The MVP matches expressions and visible attributes; it does not prove that a meme character physically resembles a student or identify a person. True visual resemblance is a separate, more advanced phase.

## Project at a glance

| Item | Current plan |
|---|---|
| Primary use | Fun, interactive college activity: selfie in, meme out. |
| Dataset | Approximately 6,000+ meme images; the exact count and total size should be verified from S3. |
| Current S3 bucket | my-meme-database-2026 |
| Existing prefixes | images/ and metadata/ |
| Region | us-east-1 (N. Virginia), based on the S3 location command. |
| MVP matching | Rule-based weighted scoring from facial attributes, detected labels, and optional human-assigned tags. |
| Target usage | Planning assumption: 500–1,000 students; peak concurrent traffic must be measured, not assumed. |
| Status | Dataset upload is complete according to the project notes; preprocessing and application deployment remain pending. |

# 2. Problem Statement

College events and student communities often use memes and photos for entertainment, but manually selecting a meme for each participant is repetitive and does not create an interactive, personalized experience. The project proposes a secure web platform that accepts a voluntary student selfie, analyzes visual expression cues, and returns a relevant meme from a large curated dataset. The platform demonstrates a practical, low-maintenance serverless architecture using AWS object storage, facial analysis, API routing, event/backend compute, access control, and monitoring.

# 3. Goals, Scope, and Non-Goals

## Goals for the MVP

- Allow a student to capture or select an image in a browser.
- Upload the image directly to private S3 using a short-lived presigned URL; do not expose AWS credentials to the browser.

- Analyze each submitted selfie once per submission using Rekognition DetectFaces.
- Precompute catalogue attributes for memes once, persist metadata, and never re-analyze all memes for every student request.
- Rank candidate memes with a transparent weighted scoring algorithm and choose among the top candidates for variety.
- Return the selected meme using a short-lived presigned GET URL.
- Keep uploaded student images private and delete them automatically after a short retention period unless there is a documented reason to keep them.

### Out of scope for the first version

- Confirming a person's identity or building a face-recognition attendance system.
- Claiming to infer a person's actual mood, personality, gender identity, or other sensitive traits from a photo.
- Automatically understanding every joke, sarcasm, cultural reference, or caption in every meme.
- Training a custom computer-vision model before the baseline is tested.
- Building a recommendation history, social feed, likes, comments, or analytics dashboard unless the MVP needs them.

# 4. System Architecture

## 4.1 One-time meme-catalogue preprocessing

This pipeline runs once for the current dataset and again only when new memes are added or the tagging logic changes. The initial implementation can run from AWS CloudShell with Python/boto3. It should list objects with pagination, filter supported image formats, throttle requests, retry throttling errors, checkpoint progress, and save a metadata file to S3.

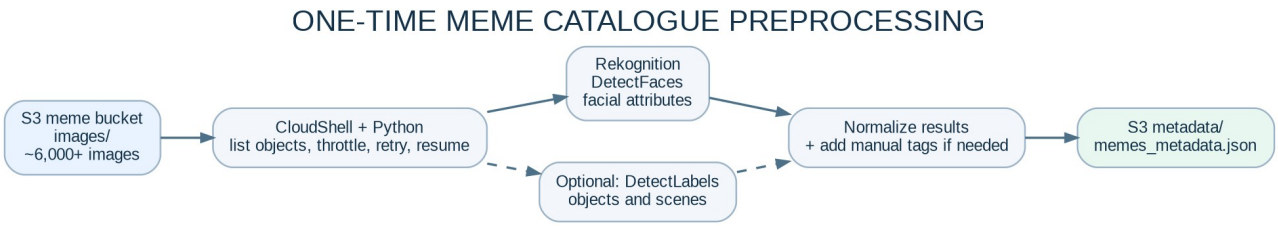



**Figure 1. One-time dataset pipeline. DetectLabels is optional in the first iteration and should be enabled only if object/scene tags improve matching.**

| Stage | What happens | Output |
|---|---|---|
| 1. List | Read every image object under images/; verify count and extensions. | List of S3 keys |
| 2. Detect faces | Call DetectFaces with Attributes=ALL on each meme image. | Face count, facial attributes, emotion scores where returned |
| 3. Optional labels | Call DetectLabels on a sample or all images if object/scene labels are useful. | Labels such as person, phone, indoor/outdoor |
| 4. Normalize | Convert service responses to a small stable schema. Add manual tags separately if needed. | Consistent metadata records |
| 5. Persist | Write a resumable local JSON file, then upload it to metadata/. | metadata/memes_metadata.json |

**COST CONTROL** Start with a 20–50 image trial. Check response quality, throttling, and current charges before running the full dataset. Each separate Rekognition API call against an image is billed as a separate analysis; running both DetectFaces and DetectLabels roughly doubles the analysis-call count.

## 4.2 Live student request

The runtime path is different from preprocessing. It never loops over all 6,000+ source images by calling Rekognition. It analyzes the new student image, compares its features with the saved metadata, then returns the selected catalogue image.

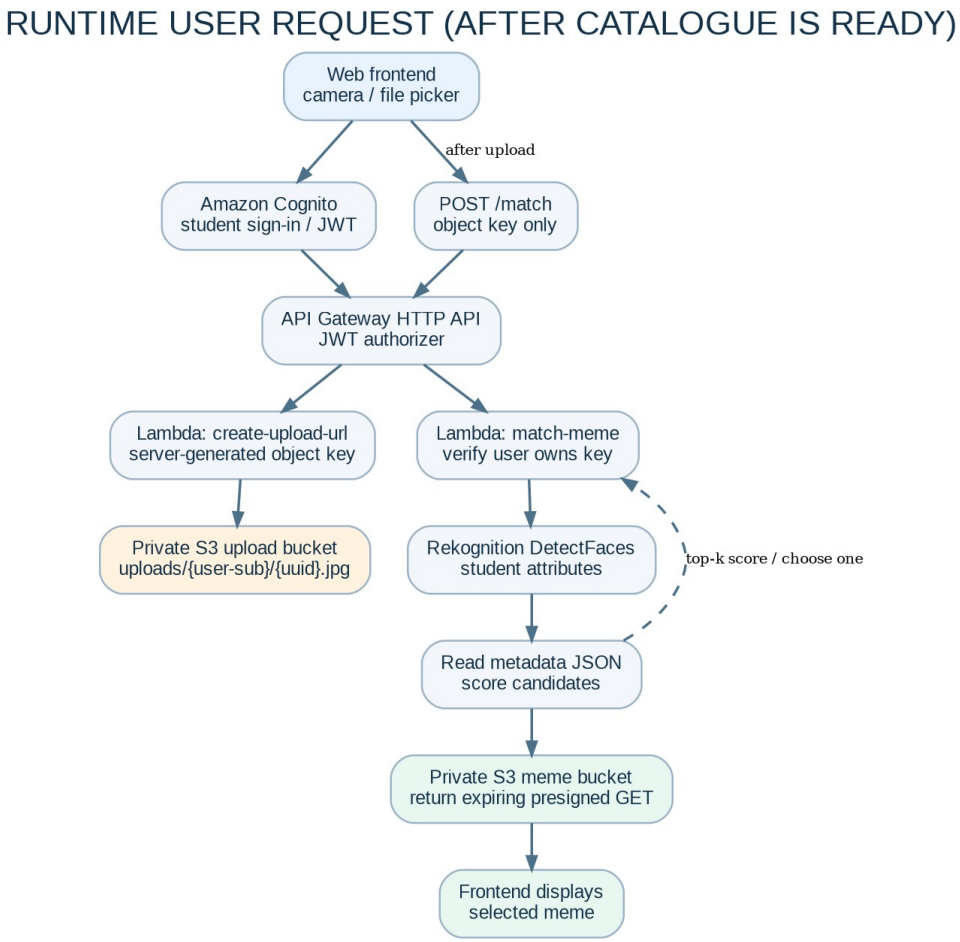



**Figure 2. Proposed runtime flow. Cognito provides identity; API Gateway and Lambda validate requests; S3 remains private.**

| Step | Component interaction |
|---|---|
| 1. Sign in | The student signs in using Amazon Cognito; the browser receives an identity token for API requests. |
| 2. Ask for upload URL | Frontend calls POST /upload-url through API Gateway. Lambda generates a random object key tied to the authenticated user and returns a time-limited presigned PUT URL. |
| 3. Upload directly | Browser uploads the image directly to a private student-upload bucket. The image bytes do not pass through API Gateway. |
| 4. Request matching | Browser calls POST /match with the object key and its valid |

| Step | Component interaction |
|---|---|
| | authentication token. |
| 5. Validate ownership | Lambda verifies the authenticated user owns that key. It must not trust arbitrary bucket/key values from the client. |
| 6. Analyze selfie | Lambda invokes Rekognition DetectFaces once for this submission and checks that a usable face is present. |
| 7. Score catalogue | Lambda loads the precomputed metadata and ranks candidates using weighted attribute similarity. It randomly selects from a top-k shortlist to create variety. |
| 8. Return result | Lambda creates a short-lived presigned GET URL for the chosen meme and returns it to the browser, which renders the result. |
| 9. Delete later | S3 Lifecycle expires student uploads after the chosen retention window; an explicit delete may be performed after a successful result as well. |

# 5. AWS Services and Their Responsibilities

| Service | Role in the project | MVP decision |
|---|---|---|
| Amazon S3 | Stores the meme catalogue, precomputed metadata, and temporary student uploads. Supports private objects and presigned URLs. | Required. Keep meme catalogue and student uploads in separate buckets. |
| Amazon Rekognition | DetectFaces provides face presence and attributes; DetectLabels provides generic objects/scenes if enabled. | Required for expression-based MVP; start with DetectFaces. |
| AWS Lambda | Runs upload-URL and match endpoints; executes the one-time preprocessing script separately in CloudShell. | Required for runtime. Do not run the entire dataset in one Lambda invocation. |
| Amazon API Gateway HTTP API | Exposes HTTPS API routes and connects requests to Lambda. | Required. |
| Amazon Cognito User Pools | Student authentication; JWT authorizer on API Gateway. Allows the backend to associate uploads with an authenticated identity. | Strongly recommended before multi-student testing. |
| AWS IAM | Grants least-privilege access to required S3 prefixes and Rekognition APIs. | Required. |
| Amazon CloudWatch | Lambda logs, error investigation, latency and invocation metrics. | Required for troubleshooting. |
| AWS Budgets / billing alerts | Alerts the owner when spending reaches configured thresholds. Alerts are not a guaranteed hard stop. | Set up before batch processing. |
| DynamoDB | Could store match history, per-user state, or catalog entries if the system outgrows a JSON file. | Not required for the first MVP. |
| CloudFront / Amplify Hosting | Can host the frontend. Keep website hosting separate from the private image bucket. | Choose later; initial UI can run locally while the API is tested. |

# 6. Matching and Classification Logic

## 6.1 What AWS can label automatically

- DetectFaces: face count, bounding boxes, confidence, facial attributes, pose, smile and emotion scores when available.
- DetectLabels: generic objects, concepts and scenes (for example, person, phone or outdoor setting).
- Neither standard API reliably understands that a meme is funny, sarcastic, relatable, romantic, or culturally appropriate.

**DO NOT CONFUSE THE TERMS** Face detection/attribute analysis finds visible faces and observable attributes. Face recognition or identity matching is a different capability. This MVP does not identify students or prove a meme character looks like them.

## 6.2 Catalogue metadata

Each meme record should include its S3 key and normalized analysis results. If the source image contains multiple faces, keep a face list and define a deterministic comparison policy (for example, compare the largest detected face, while retaining face_count). Images with no detected face still remain in the catalogue and may use object labels or manual tags as fallback candidates.

```
{
 "schema_version": "1.0",
 "generated_at": "2026-10-26T00:00:00Z",
 "items": [
  {
   "key": "images/image_1.jpg",
   "face_count": 1,
   "faces": [{
    "top_emotion": "HAPPY",
    "emotion_confidence": 91.4,
    "smile": true,
    "eyeglasses": false,
    "mouth_open": true
   }],
   "labels": ["Person", "Phone"],
   "manual_tags": ["reaction"],
   "status": "success"
  }
 ]
}
```

*Example only; real attribute fields must be normalized safely because an attribute may be absent, have a low confidence, or be unavailable for an image.*

## 6.3 Recommended MVP scoring

Use a transparent rule-based score first. For a single prominent face, one possible starting score is:

```
score = 0.50 * emotion_similarity
      + 0.20 * smile_match
      + 0.10 * mouth_open_match
      + 0.10 * glasses_match
      + 0.10 * relevant_tag_overlap
```

Each component should be normalized to 0–1. If an attribute is missing or confidence is low, omit or down-weight that component rather than treating missing as false. Tune the weights against a small human-reviewed test set; the values above are initial heuristics, not validated scientific weights.

- Rank the meme catalogue by score.
- Keep the top 3–10 candidates (top-k).
- Choose randomly or with score-weighted sampling among that shortlist to avoid repeating one meme every time.
- If the student image contains no usable face, show a friendly error or use a non-face fallback rather than inventing attributes.
- For stronger semantic categories such as sarcasm or “same energy,” add human-assigned tags or a separate vision-language/embedding approach later.

### 6.4 Two different product ambitions

| Goal | Approach | Status |
|---|---|---|
| “My expression matches this meme” | Compare visible expressions and optional attributes using the preprocessed catalogue. | Recommended MVP. |
| “This meme character physically resembles my face” | Needs a separate visual-similarity method, candidate face representations/embeddings, or carefully designed face comparison. It is not achieved by comparing emotion labels. | Future research phase; review privacy and fairness before implementation. |
| “This meme is funny or sarcastic” | Need semantic understanding of captions/context, human tags, OCR/text analysis, or a suitable vision-language model. Standard DetectFaces/DetectLabels alone is insufficient. | Future improvement. |

## 7. API Contract

The frontend should not receive AWS access keys and should not send raw image bytes through API Gateway. The backend generates constrained presigned URLs, while S3 carries the image transfer directly.

| Route | Request (simplified) | Response (simplified) | Purpose |
|---|---|---|---|
| POST /upload-url | { "contentType": "image/jpeg" } | { "uploadUrl": "...", "objectKey": "uploads/{sub}/{uuid}.jpg", "expiresIn": 300 } | Create a short-lived direct-to-S3 upload permission. |
| POST /match | { "objectKey": "uploads/{sub}/{uuid}.jpg" } | { "memeUrl": "...", "score": 0.91, "matchId": "..." } | Validate key ownership, analyze user image, score catalogue, return chosen meme URL. |
| GET /health (optional) | No body | { "status": "ok" } | Basic API health check for demo/testing. |

**SECURITY RULE** Lambda must derive the user identity from the verified Cognito JWT, generate the object key server-side, and verify the key belongs to that user before reading it. A browser-provided object key is not proof of ownership.

### How the API connects to the backend

1. Create an API Gateway HTTP API in the same chosen AWS Region.
2. Create the Lambda functions and attach execution roles with only the permissions required by each function.
3. Configure POST /upload-url and POST /match routes with Lambda integrations.
4. Configure a Cognito JWT authorizer for those routes.
5. Enable CORS for the exact frontend origin, not a wildcard in production unless justified.
6. Frontend calls the API over HTTPS using fetch() with the Authorization: Bearer <token> header.
7. Add CloudWatch logs, request validation, image type/size limits, and consistent error responses.

## 8. Security, Privacy, and Responsible Use

| Control | Required implementation |
|---|---|
| Private storage | Keep S3 Block Public Access enabled. Meme images and student uploads remain private; return time-limited presigned URLs for specific objects only. |
| Separate buckets | Keep the existing meme/metadata bucket separate from a new student-upload bucket. This reduces accidental exposure and simplifies lifecycle rules. |
| Authentication | Use Cognito for a multi-student release and a JWT authorizer in API Gateway. |
| Authorization | Use IAM least privilege. Students never get AWS credentials or permission to list the bucket. Lambda checks the authenticated user/key relationship. |
| Retention | Apply an S3 Lifecycle rule such as expiry after 1 day for temporary uploads, then test it. Delete sooner after processing if the experience allows it. |
| Consent and notice | Tell students what is analyzed, why it is analyzed, who administers the platform, how long photos are retained, and how to request deletion. Obtain appropriate consent and institution approval before a real campus launch. |
| AI service data use | Before submitting student faces, review AWS Rekognition data-use and AI services opt-out settings/policies for the account. The AWS documentation says images submitted to Rekognition may be stored and used to improve the service unless opted out. |
| Avoid sensitive classification | Do not use skin-tone categories such as “fair/dark” or infer identity, personality, or internal emotional state. Base the MVP on limited visible expression cues and allow a neutral/fallback result. |
| Monitoring | Log request IDs, success/failure codes and processing time. Avoid logging raw images, presigned URLs, full tokens, or unnecessary face attributes. |
| Region decision | Current meme bucket is in us-east-1. Before processing actual student images, confirm data residency, latency, institutional rules and the intended AWS Region. Rekognition S3-object input requires the S3 object and Rekognition operation to use the same Region. |

# 9. Cost and Scalability Plan

## 9.1 Main cost drivers

- S3 storage for about 0.7 GB of meme images is expected to be small; S3 request, data-transfer and related charges still apply.
- Rekognition image analysis is the key variable cost. Each API call is billed as an image analysis; calling DetectFaces and DetectLabels on the same image counts as two analyses.
- Lambda and API Gateway costs depend on request volume, execution time, memory and API type. CloudWatch log volume also matters.
- Presigned URLs themselves do not make an object public; S3 GET/PUT requests and data transfer may incur charges.
- A $150 credit balance is a budget, not proof that every service is free. Track actual billing and region-specific prices.

## 9.2 Illustrative preprocessing estimate

AWS currently shows an example first-tier price of about US$0.001 per image for common Rekognition image APIs, with tiered pricing and regional/account details. Using that example only: 6,692 images × one API ≈ US$6.69; two APIs per image ≈ US$13.38. These are planning estimates, not a quote. Verify the current pricing page and billing dashboard before the full run, and check free-tier/credit eligibility separately.

## 9.3 Practical guardrails

- Create a small AWS Budget and email alerts before full preprocessing. Alerts notify but do not guarantee an automatic hard stop.
- Process a test batch first and record the number of successful and failed objects.
- Use bounded concurrency, retries with exponential backoff, and checkpointing. Avoid a large uncontrolled burst of API calls.
- Persist metadata and resume from successful entries after a failure; do not reprocess already-completed images unnecessarily.
- Run DetectLabels only if its tags measurably help matching; the MVP can begin with DetectFaces plus manually assigned semantic tags.
- Keep a count of API calls and failures in the preprocessing log so the run is auditable.

# 10. Implementation Roadmap

| Phase | Tasks | Completion evidence |
|---|---|---|
| 0. Verify current data | Confirm exact image count/bytes, supported image extensions, bucket region, Block Public Access and existing object layout. | S3 summary recorded; no API spend yet. |
| 1. Single-image smoke test | Use CloudShell and boto3 to call DetectFaces on one image from images/. Confirm bucket and Rekognition Region match and IAM permissions work. | Successful response with face count/attributes or a documented no-face result. |
| 2. Small preprocessing batch | Process 20–50 images; test metadata schema, errors, request rate and approximate cost. | Sample JSON reviewed and uploaded to metadata/. |
| 3. Full catalogue preprocessing | Run resumable script across remaining images. Decide whether DetectLabels is needed. Upload final metadata file. | All eligible image keys have success/error status; counts reconcile. |

| Phase | Tasks | Completion evidence |
|---|---|---|
| 4. Matching function | Implement metadata loading, normalized attribute extraction, weighted scoring, top-k selection and fallback behaviour. | Unit tests for smile/emotion matches, no-face image, multiple faces and missing attributes. |
| 5. API and security | Create Cognito, HTTP API, upload-URL Lambda, match Lambda, IAM roles and CORS rules. | Authenticated upload and match requests work; user cannot access another user's key. |
| 6. Frontend | Create camera/file-upload UI, loading state, error state, result display, consent notice and retry action. | End-to-end browser demo on desktop and phone. |
| 7. Cost/privacy readiness | Enable budgets/alerts, CloudWatch logs, S3 lifecycle expiration and review AI-service data-use settings. | Documented privacy notice, retention test and cost check. |
| 8. Campus pilot | Pilot with a small opt-in group, gather feedback, adjust weights and only then increase concurrent use. | Measured latency, error rate, API consumption and user feedback. |

# 11. Test Plan and Acceptance Criteria

| Test case | Expected result |
|---|---|
| Valid JPEG/PNG with one face | Face attributes are extracted and a meme URL is returned. |
| No face / obscured face | Clear fallback message or configured fallback selection; no crash. |
| Multiple faces | Largest-face comparison policy applied consistently; face count recorded. |
| Meme has no detectable face | Metadata marks zero faces; object/manual tags may still permit fallback selection. |
| Missing attribute / low confidence | Scoring skips or down-weights the field rather than treating missing as false. |
| Corrupt/unsupported image | Request rejected with a safe message; image not repeatedly analyzed. |
| Different user submits another user's key | Request denied; no signed URL is issued. |
| Expired presigned URL | Upload/download fails as intended; user can request a new URL. |
| Repeated request for same selfie | Each submission analyzes only the submitted selfie; catalogue images are not re-analyzed. |
| Rekognition throttling or transient error | Retries/backoff work; progress checkpoint is preserved; partial failure is reported. |
| Retention expiration | Temporary uploads expire according to configured lifecycle policy. |

## MVP acceptance criteria

- The catalogue preprocessing job produces a valid metadata file with one status record per eligible image key.
- The live path makes one DetectFaces call per valid selfie submission, not thousands of calls against the meme library.
- The frontend can upload directly to S3 and display a returned meme.
- All S3 buckets remain private; student-to-student access checks pass.
- The system handles no-face, failed-analysis, invalid-upload and API-error paths gracefully.
- Costs, retention settings and current AWS region are recorded before the student pilot.

# 12. Known Limitations and Future Enhancements

| Limitation | Possible next step |
|---|---|
| Rekognition can miss faces in cartoons, low-quality memes, side profiles or obscured faces. | Keep zero-face records; use general labels or manual tags as fallback. |
| Visible expression is not the same as a person's actual mood. | Describe the feature as expression-based meme matching, not emotion diagnosis. |
| Automatic object labels do not understand humour or sarcasm. | Add a human-labeled tag workflow, OCR for captions, or evaluate a vision-language model later. |
| Attribute matching is not physical face resemblance. | Explore image embeddings or a carefully scoped face-similarity experiment separately; review consent/privacy and cost. |
| A single JSON catalogue is simple but eventually may need faster filtering. | Consider DynamoDB for structured tags or a vector index only after measuring real bottlenecks. |
| Campus traffic may spike at events. | Measure request rate, tune Lambda/API quotas, rate-limit submissions, and consider asynchronous queues if processing becomes bursty. |

# 13. Ready-to-Use Project Explanation

## 30-second summary

**PITCH** "We are building an AWS-based Image-to-Meme Matching Platform for college students. A student captures or uploads a selfie, and the backend uses Amazon Rekognition to detect visible facial attributes and expressions. We analyze our meme catalogue once and store the results as metadata in S3. For every new selfie, our matching algorithm compares its attributes with that saved catalogue, ranks the best candidates, and returns a meme through a secure, temporary URL. The architecture uses S3, Lambda, API Gateway, Cognito, IAM, Rekognition and CloudWatch, with privacy and cost controls built in."

## Likely viva / review questions

| Question | Suggested answer |
|---|---|
| Why AWS serverless? | It avoids maintaining always-on servers and lets the project use managed storage, API routing, compute, authentication and image analysis. Costs mostly scale with actual requests, though all service charges must still be monitored. |
| Why preprocess memes? | The meme dataset changes infrequently. Computing metadata once prevents repeated Rekognition calls on thousands of memes for each student request. |
| Why not classify the student and all memes in one request? | That would repeat unnecessary work and increase latency/cost. The catalogue attributes are persisted; only the newly uploaded selfie is analyzed at runtime. |
| What algorithm is used? | The MVP uses weighted rule-based similarity scoring and top-k randomized selection, not a trained custom model. |
| Is this face recognition? | The initial feature is face detection and visible attribute analysis, not identity recognition. Physical resemblance requires a different method. |
| How do you protect user data? | Private buckets, Cognito authentication, API JWT authorization, server-generated keys, least-privilege IAM, short-lived presigned URLs, short retention, and no client AWS keys. |
| Can it recognize sarcasm? | Not reliably with DetectFaces/DetectLabels alone. Human tags or a semantic vision-language approach would be needed. |

| Question | Suggested answer |
|---|---|
| How do you control costs? | Test a small sample, track API calls, use the minimum necessary Rekognition APIs, preprocess once, set budget alerts, throttle/retry, and measure usage before the pilot. |

# 14. Immediate Next Actions (Current Project State)

1. Verify exact object count and total bytes using the S3 recursive summary. Do not use "| head" for this check because it can produce a harmless Broken pipe message.

```
aws s3 ls s3://my-meme-database-2026/images/ --recursive --summarize
```

2. Confirm the current caller identity and region context before granting permissions or running billed calls.

```
aws sts get-caller-identity
aws configure list
```

3. Run a one-image DetectFaces smoke test with Rekognition in us-east-1. The bucket location command returned LocationConstraint=null, which indicates the default S3 Region us-east-1 for this general-purpose bucket. Rekognition and the S3 object input must be in the same Region.
4. If that succeeds, run a small 20–50 image preprocessing test and inspect the JSON carefully. Only then run the complete dataset job with bounded concurrency, retries and checkpointing.
5. Upload the finished metadata JSON to s3://my-meme-database-2026/metadata/memes_metadata.json. The metadata/ prefix currently exists, but the analysis output is not yet confirmed to exist.
6. After the catalogue is ready, build the API and frontend in this order: Cognito → IAM roles → upload-url Lambda → API Gateway route → private student-upload bucket → match Lambda → frontend → CloudWatch/budget/lifecycle controls.

**CURRENT STATUS CHECK** The last confirmed state is that images/ and metadata/ exist in the S3 bucket and a CLI listing returned example image keys. No successful Rekognition test or completed metadata file has yet been confirmed in this project record.

# 15. Official AWS References

These official pages support the service behaviours and implementation decisions described in this report. Verify current pricing and account-specific settings before deployment.

1. **Amazon Rekognition DetectFaces:** Amazon Rekognition DetectFaces — Face detection and supported facial attributes.
2. **Amazon Rekognition DetectLabels:** Amazon Rekognition DetectLabels — Object, concept, scene and image-property labels.
3. **Analyze images stored in S3:** Analyze images stored in S3 — S3 object input; bucket and Rekognition region must match.
4. **Amazon Rekognition pricing:** Amazon Rekognition pricing — Per-image analysis pricing; multiple APIs against one image count as multiple analyses.
5. **S3 presigned URLs:** S3 presigned URLs — Temporary upload/download access without exposing AWS credentials.
6. **Cognito + API Gateway:** Cognito + API Gateway — Using Cognito tokens and API Gateway authorizers to protect API resources.

7. **S3 default encryption FAQ:** S3 default encryption FAQ — SSE-S3 default encryption for new objects at no extra charge.
8. **AWS AI services opt-out policies:** AWS AI services opt-out policies — Policy to opt out of content storage/use for supported AI services; check account applicability.
9. **Rekognition service quotas:** Rekognition service quotas — Region-specific API quotas and TPS guidance.
10. **AWS Lambda quotas:** AWS Lambda quotas — Lambda execution limits, including the 15-minute standard timeout maximum.
11. **AWS CloudShell:** AWS CloudShell — Browser-based shell with AWS CLI and console credentials.

*End of report. This is an implementation blueprint, not a claim that all components have already been deployed.*